

Sentinel AI Security Audit Report

Audit Metadata

- **Filename:** vuln.elf
- **SHA-256 Checksum:** d9d22f14657549c42c6c72738fa661101f5edf50b7ac19f1ff5cd48fedc06da7
- **Verdict:** VULNERABLE
- **Risk Level:** CRITICAL
- **Total Functions Evaluated:** 5

Executive Summary

The security audit of `vuln.elf` has returned a **VULNERABLE** verdict with a **CRITICAL** risk rating. Our proprietary GNN-based analysis engine, GATv2, identified a severe memory safety violation within the core processing logic. The primary threat vector involves a classic stack-based buffer overflow that bypasses logical boundaries, potentially allowing for arbitrary code execution (ACE) or a complete system compromise.

Given the nature of the vulnerability, immediate remediation is required. While the binary includes stack-smashing protection (canaries), the underlying logic violates fundamental SEI CERT C security standards, making the application unstable and exploitable under specific conditions.

Analysis Metrics

- **Static Analysis Engine:** GNN Compiler Wrapper (GATv2 Architecture)
- **Total Functions Scanned:** 5
- **Flagged Functions:** 1
- **Model Confidence Score:** 85.00%
- **Analysis Depth:** Control Flow Graph (CFG) & Instruction-Level Heuristics

Detailed Findings

Finding 1: Function `vuln\_c\_func`

- **Identified Threat:** CWE-119 (Improper Restriction of Operations within the Bounds of a Memory Buffer) / CWE-787 (Out-of-bounds Write)
- **Model Confidence:** 85.00%

Technical Breakdown of Exploit Vector:

The disassembly reveals a critical failure in memory management within `Basic Block 1`.

1. **Stack Allocation:** The function allocates a local buffer on the stack (at `RBP - 0x20`).
2. **Unbounded Copy:** The function retrieves a pointer from `RDI` (the first argument) and passes it directly as the source (`RSI`) to the `strcpy` function. The destination (`RDI`) is the local stack buffer.
3. **The Vulnerability:** `strcpy` does not perform any length verification. If the input string pointed to by `RDI` exceeds the

allocated 32-byte space (0x20), it will overwrite adjacent stack memory.

4. **Impact:** This overwrite targets the saved Frame Pointer, the Return Address, and the Stack Canary. Although `Basic Block 4` shows a call to `\_\_stack\_chk\_fail`, an attacker can still cause a Denial of Service (DoS) or, in complex scenarios, leverage the overflow before the canary check occurs.

Violated Security Standards:

- **CWE-787:** Out-of-bounds Write.
- **CERT C Rule ARR30-C:** Do not form or use out-of-bounds pointers or array subscripts.
- **CERT C Rule STR31-C:** Guarantee that storage for strings has sufficient space for character data and the null terminator.

Remediation & Secure Code Fix:

The use of `strcpy` must be deprecated in favor of bounds-checked alternatives. Below is the secure implementation using `strncpy` and manual null-termination to prevent buffer overruns.

```
/* Secure Remediation of vuln_c_func */
#include <string.h>
#include <stdio.h>

#define BUFFER_SIZE 32

void secure_c_func(const char *input) {
    char dest[BUFFER_SIZE];

    if (input == NULL) {
        return;
    }

    // SECURE: Use strncpy to limit the copy to the size of the destination buffer
    // Note: strncpy does not guarantee null-termination if the source is longer than the limit
    strncpy(dest, input, BUFFER_SIZE - 1);

    // Ensure null-termination
    dest[BUFFER_SIZE - 1] = '\0';

    printf("Secure_C: %s\n", dest);
}
```

General Mitigations & Best Practices

To harden the application against memory corruption exploits, the following systemic mitigations are recommended:

1. **Adopt Secure API Wrappers:** Prohibit the use of "dangerous" functions such as `strcpy`, `gets`, and `sprintf`. Replace them with `strncpy`, `fgets`, and `snprintf` respectively.
2. **Compiler Hardening Flags:**
 - **Stack Canaries:** Ensure `-fstack-protector-all` is enabled (detected in current binary, but logic remains flawed).
 - **ASLR/PIE:** Compile with `-fPIE -pie` to ensure Position Independent Executable status, making Return-Oriented Programming (ROP) significantly harder.
 - **NX/DEP:** Ensure the stack is marked as non-executable (`-z noexecstack`) to prevent shellcode execution.
3. **Static & Dynamic Analysis (SAST/DAST):** Integrate automated GNN-based auditors (like Sentinel AI) into the CI/CD pipeline to catch memory safety issues before binary compilation.
4. **Address Sanitizer (ASan):** During the testing phase, compile with `-fsanitize=address` to detect out-of-bounds

accesses at runtime.

End of Report

Sentinel AI Security Division